

Case Study: **PORTER - NEURAL NETWORK ~** Regression

Analysed by : **KASI**

About Project

Objective

Porter is India's Largest Marketplace for Intra-City Logistics. Leader in the country's \$40 billion intra-city logistics market, Porter strives to improve the lives of 1,50,000+ driver-partners by providing them with consistent earning & independence. Currently, the company has serviced 5+ million customers

Porter works with a wide range of restaurants for delivering their items directly to the people.

Porter has a number of delivery partners available for delivering the food, from various restaurants and wants to get an estimated delivery time that it can provide the customers on the basis of what they are ordering, from where and also the delivery partners.

This dataset has the required data to train a regression model that will do the delivery time estimation, based on all those features

Data Dictionary

Each row in this file corresponds to one unique delivery. Each column corresponds to a feature as explained below.

- market_id : integer id for the market where the restaurant lies
- created_at : the timestamp at which the order was placed
- actual_delivery_time : the timestamp when the order was delivered
- store_primary_category : category for the restaurant
- order_protocol : integer code value for order protocol(how the order was placed ie: through porter, call to restaurant, pre booked, third part etc)
- total_items subtotal : final price of the order
- num_distinct_items : the number of distinct items in the order
- min_item_price : price of the cheapest item in the order
- max_item_price : price of the costliest item in order
- total_onshift_partners : number of delivery partners on duty at the time order was placed
- total_busy_partners : number of delivery partners attending to other tasks
- total_outstanding_orders : total number of orders to be fulfilled at the moment

Let's Get Started

Importing Libraries

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objs as go
import plotly.subplots as sp
from plotly.subplots import make_subplots
import plotly.express as px
import warnings
import time
import datetime as dt
import tensorflow as tf
import keras
from keras import layers
from tensorflow.keras import Model
from tensorflow.keras import Sequential
from tensorflow.keras.optimizers import Adam
warnings.filterwarnings("ignore")
```

```
In [2]: !pip install --upgrade plotly
```

Requirement already satisfied: plotly in /usr/local/lib/python3.10/dist-packages (5.24.1)
Collecting plotly
 Downloading plotly-6.0.1-py3-none-any.whl.metadata (6.7 kB)
Requirement already satisfied: narwhals>=1.15.1 in /usr/local/lib/python3.10/dist-packages (from plotly) (1.18.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from plotly) (24.2)
Downloading plotly-6.0.1-py3-none-any.whl (14.8 MB)
----- 14.8/14.8 MB 87.5 MB/s eta 0:00:00
Installing collected packages: plotly
 Attempting uninstall: plotly
 Found existing installation: plotly 5.24.1
 Uninstalling plotly-5.24.1:
 Successfully uninstalled plotly-5.24.1
Successfully installed plotly-6.0.1

Importing Dataset

```
In [3]: df = pd.read_csv('/kaggle/input/porter-dataset/dataset.csv')
df
```

Out[3]:

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	n
0	1.0	2015-02-06 22:24:17	2015-02-06 23:27:16	df263d996281d984952c07998dc54358	american	1.0	4	3441	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:56:29	f0ade77b43923b38237db569b016ba25	mexican	2.0	1	1900	
2	3.0	2015-01-22 20:39:28	2015-01-22 21:09:09	f0ade77b43923b38237db569b016ba25	NaN	1.0	1	1900	
3	3.0	2015-02-03 21:21:45	2015-02-03 22:13:00	f0ade77b43923b38237db569b016ba25	NaN	1.0	6	6900	
4	3.0	2015-02-15 02:40:36	2015-02-15 03:20:26	f0ade77b43923b38237db569b016ba25	NaN	1.0	3	3900	
...	...	...	...	...	...	...	...	...	...
197423	1.0	2015-02-17 00:19:41	2015-02-17 01:24:48	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	3	1389	
197424	1.0	2015-02-13 00:01:59	2015-02-13 00:58:22	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	6	3010	
197425	1.0	2015-01-24 04:46:08	2015-01-24 05:36:16	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	5	1836	
197426	1.0	2015-02-01 18:18:15	2015-02-01 19:23:22	c81e155d85dae5430a8cee6f2242e82c	sandwich	1.0	1	1175	
197427	1.0	2015-02-08 19:24:33	2015-02-08 20:01:41	c81e155d85dae5430a8cee6f2242e82c	sandwich	1.0	4	2605	

197428 rows × 14 columns

Custom Color Palette

```
In [4]: def custom_palette(custom_colors):
customPalette = sns.set_palette(sns.color_palette(custom_colors))
sns.palplot(sns.color_palette(custom_colors),size=0.8)
plt.tick_params(axis='both', labelsize=0, length = 0)
```

```
In [5]: #defining colour palette
red = ["#4f000b", "#720026", "#ce4257", "#ff7f51", "#ff9b54"]
bo = ["#6930c3", "#5e60ce", "#0096c7", "#48cae4", "#ade8f4", "#ff7f51", "#ff9b54", "#ffbf69"]
pink = ["#aa4465", "#dd2d4a", "#f26a8d", "#f49cbb", "#ffcbbf", "#e2afff", "#ff86c8", "#ffa3a5", "#ffbf81", "#e9b827", "#f9e576"]
custom_palette(pink)
```



Basic Analysis

Shape of the Data

```
In [6]: df.shape
```

Out[6]: (197428, 14)

Inference:

- Shape of dataframe is 197428 * 14
- Number of rows = 197428
- Number of columns = 14

First 5 Rows

```
In [7]: df.head()
```

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_di
0	1.0	2015-02-06 22:24:17	2015-02-06 23:27:16	df263d996281d984952c07998dc54358	american	1.0	4	3441	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:56:29	f0ade77b43923b38237db569b016ba25	mexican	2.0	1	1900	
2	3.0	2015-01-22 20:39:28	2015-01-22 21:09:09	f0ade77b43923b38237db569b016ba25	NaN	1.0	1	1900	
3	3.0	2015-02-03 21:21:45	2015-02-03 22:13:00	f0ade77b43923b38237db569b016ba25	NaN	1.0	6	6900	
4	3.0	2015-02-15 02:40:36	2015-02-15 03:20:26	f0ade77b43923b38237db569b016ba25	NaN	1.0	3	3900	

Columns in a Dataframe

```
In [8]: df.columns
```

Out[8]: Index(['market_id', 'created_at', 'actual_delivery_time', 'store_id', 'store_primary_category', 'order_protocol', 'total_items', 'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price', 'total_onshift_partners', 'total_busy_partners', 'total_outstanding_orders'], dtype='object')

Concise Summary

```
In [9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 197428 entries, 0 to 197427
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   market_id                            196441 non-null float64
1   created_at                           197428 non-null object
2   actual_delivery_time                  197421 non-null object
3   store_id                             197428 non-null object
4   store_primary_category                192668 non-null object
5   order_protocol                       196433 non-null float64
6   total_items                          197428 non-null int64
7   subtotal                             197428 non-null int64
8   num_distinct_items                   197428 non-null int64
9   min_item_price                       197428 non-null int64
10  max_item_price                       197428 non-null int64
11  total_onshift_partners                181166 non-null float64
12  total_busy_partners                  181166 non-null float64
13  total_outstanding_orders              181166 non-null float64
dtypes: float64(5), int64(5), object(4)
memory usage: 21.1+ MB
```

Inference:

The columns 'market_id', 'actual_delivery_time', 'store_primary_category', 'order_protocol', 'total_onshift_partners', 'total_busy_partners', and 'total_outstanding_orders' have missing values.

Statistical Summary

```
In [10]: # Statistical Summary of numeric Columns
df.describe().T
```

Out[10]:

	count	mean	std	min	25%	50%	75%	max
market_id	196441.0	2.978706	1.524867	1.0	2.0	3.0	4.0	6.0
order_protocol	196433.0	2.882352	1.503771	1.0	1.0	3.0	4.0	7.0
total_items	197428.0	3.196391	2.666546	1.0	2.0	3.0	4.0	411.0
subtotal	197428.0	2682.331402	1823.093688	0.0	1400.0	2200.0	3395.0	27100.0
num_distinct_items	197428.0	2.670791	1.630255	1.0	1.0	2.0	3.0	20.0
min_item_price	197428.0	686.218470	522.038648	-86.0	299.0	595.0	949.0	14700.0
max_item_price	197428.0	1159.588630	558.411377	0.0	800.0	1095.0	1395.0	14700.0
total_onshift_partners	181166.0	44.808093	34.526783	-4.0	17.0	37.0	65.0	171.0
total_busy_partners	181166.0	41.739747	32.145733	-5.0	15.0	34.0	62.0	154.0
total_outstanding_orders	181166.0	58.050065	52.661830	-6.0	17.0	41.0	85.0	285.0

In [11]:

```
# Statistical Summary of Categorical Columns
df.describe(include='object').T
```

Out[11]:

	count	unique	top	freq
created_at	197428	180985	2015-02-11 19:50:43	6
actual_delivery_time	197421	178110	2015-02-08 04:09:25	5
store_id	197428	6743	d43ab110ab2489d6b9b2caa394bf920f	937
store_primary_category	192668	74	american	19399

Checking For Missing Values

In [12]:

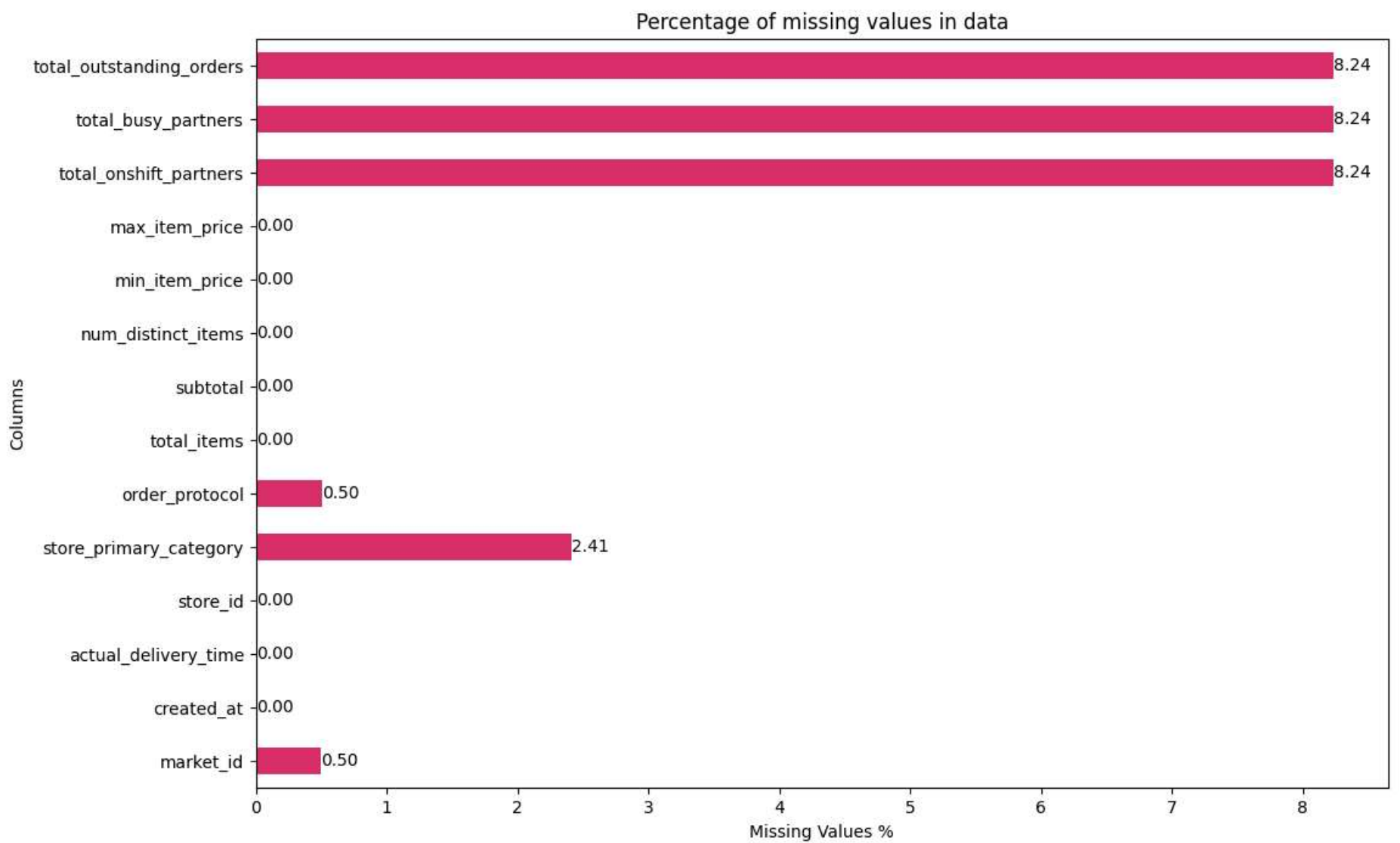
```
np.round(((df.isnull().sum())/(df.shape[0]))*100,2)
```

Out[12]:

market_id	0.50
created_at	0.00
actual_delivery_time	0.00
store_id	0.00
store_primary_category	2.41
order_protocol	0.50
total_items	0.00
subtotal	0.00
num_distinct_items	0.00
min_item_price	0.00
max_item_price	0.00
total_onshift_partners	8.24
total_busy_partners	8.24
total_outstanding_orders	8.24
dtype: float64	

In [13]:

```
plt.figure(figsize = (12,8))
ax = ((df.isnull().sum()/df.shape[0])*100).plot(kind = 'barh', color = '#DB3069')
for container in ax.containers:
    ax.bar_label(container, fmt = '%.2f')
plt.xlabel('Missing Values %')
plt.ylabel('Columns')
plt.title('Percentage of missing values in data')
sns.set_style('darkgrid')
plt.show()
```

```
In [14]: df['order_protocol'].value_counts()
```

```
Out[14]: order_protocol
1.0      54725
3.0      53199
5.0      44290
2.0      24052
4.0      19354
6.0         794
7.0         19
Name: count, dtype: int64
```

🔧 Data Preprocessing

🏠 Duplicate Value check

```
In [15]: df[df.duplicated()]
```

```
Out[15]:   market_id  created_at  actual_delivery_time  store_id  store_primary_category  order_protocol  total_items  subtotal  num_distinct_items  min_item_price
```

```
In [16]: print(df[df.duplicated()])
```

```
Empty DataFrame
Columns: [market_id, created_at, actual_delivery_time, store_id, store_primary_category, order_protocol, total_items, subtotal, num_distinct_items, min_item_price, max_item_price, total_onshift_partners, total_busy_partners, total_outstanding_orders]
Index: []
```

🔪 Handling Missing Values

Strategies to handle missing data

1 Numeric Features:

For numeric features we can use median imputation to prevent extreme values from affecting the distribution

2 Categorical/Discrete Features:

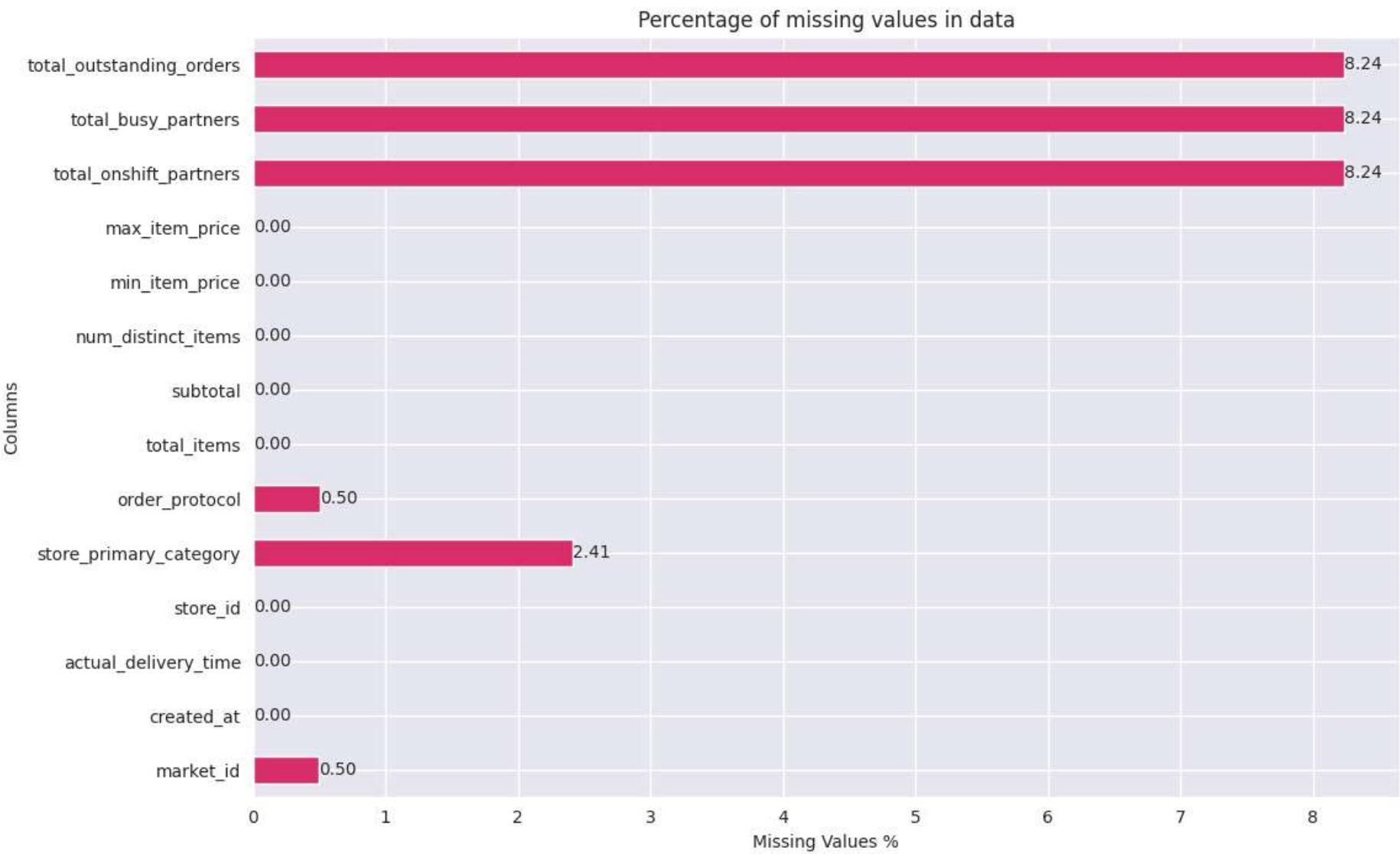
All categorical features have only 2 to 5 unique values, making it suitable to impute missing values using the most frequent category.

Missing Values Check

```
In [17]: np.round(((df.isnull().sum())/(df.shape[0]))*100,2)
```

```
Out[17]: market_id          0.50
created_at          0.00
actual_delivery_time 0.00
store_id            0.00
store_primary_category 2.41
order_protocol       0.50
total_items          0.00
subtotal            0.00
num_distinct_items   0.00
min_item_price       0.00
max_item_price       0.00
total_onshift_partners 8.24
total_busy_partners  8.24
total_outstanding_orders 8.24
dtype: float64
```

```
In [18]: plt.figure(figsize = (12,8))
ax = ((df.isnull().sum()/df.shape[0])*100).plot(kind = 'barh', color = '#DB3069')
for container in ax.containers:
    ax.bar_label(container, fmt = '%.2f')
plt.xlabel('Missing Values %')
plt.ylabel('Columns')
plt.title('Percentage of missing values in data')
sns.set_style('darkgrid')
plt.show()
```



****Handling Missing Values in 'order_protocol' Column**

```
In [19]: df['order_protocol'].skew()
```

```
Out[19]: 0.13709259426641762
```

Inference:

As column is discrete and not skewed we will use mode or most frequent value to impute missing values in the data

```
In [20]: from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='most_frequent')
df['order_protocol'] = imputer.fit_transform(df[['order_protocol']])
df.isnull().sum()
```

Out[20]: market_id 987
created_at 0
actual_delivery_time 7
store_id 0
store_primary_category 4760
order_protocol 0
total_items 0
subtotal 0
num_distinct_items 0
min_item_price 0
max_item_price 0
total_onshift_partners 16262
total_busy_partners 16262
total_outstanding_orders 16262
dtype: int64

In [21]: df

Out[21]:

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	n
0	1.0	2015-02-06 22:24:17	2015-02-06 23:27:16	df263d996281d984952c07998dc54358	american	1.0	4	3441	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:56:29	f0ade77b43923b38237db569b016ba25	mexican	2.0	1	1900	
2	3.0	2015-01-22 20:39:28	2015-01-22 21:09:09	f0ade77b43923b38237db569b016ba25	NaN	1.0	1	1900	
3	3.0	2015-02-03 21:21:45	2015-02-03 22:13:00	f0ade77b43923b38237db569b016ba25	NaN	1.0	6	6900	
4	3.0	2015-02-15 02:40:36	2015-02-15 03:20:26	f0ade77b43923b38237db569b016ba25	NaN	1.0	3	3900	
...	...	...	...	...	...	...	...	...	...
197423	1.0	2015-02-17 00:19:41	2015-02-17 01:24:48	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	3	1389	
197424	1.0	2015-02-13 00:01:59	2015-02-13 00:58:22	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	6	3010	
197425	1.0	2015-01-24 04:46:08	2015-01-24 05:36:16	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	5	1836	
197426	1.0	2015-02-01 18:18:15	2015-02-01 19:23:22	c81e155d85dae5430a8cee6f2242e82c	sandwich	1.0	1	1175	
197427	1.0	2015-02-08 19:24:33	2015-02-08 20:01:41	c81e155d85dae5430a8cee6f2242e82c	sandwich	1.0	4	2605	

197428 rows × 14 columns

Dropping columns 'market_id', 'store_id', 'store_primary_category'

Dropping the columns 'market_id', 'store_id', and 'store_primary_category' as they are not relevant to the given business context.

In [22]: df = df.drop(columns = ['market_id', 'store_id', 'store_primary_category'])

In [23]: df

Out[23]:

	created_at	actual_delivery_time	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners
0	2015-02-06 22:24:17	2015-02-06 23:27:16	1.0	4	3441	4	557	1239	33.0
1	2015-02-10 21:49:25	2015-02-10 22:56:29	2.0	1	1900	1	1400	1400	1.0
2	2015-01-22 20:39:28	2015-01-22 21:09:09	1.0	1	1900	1	1900	1900	1.0
3	2015-02-03 21:21:45	2015-02-03 22:13:00	1.0	6	6900	5	600	1800	1.0
4	2015-02-15 02:40:36	2015-02-15 03:20:26	1.0	3	3900	3	1100	1600	6.0
...	...	...	...	...	...	...	...	...	...
197423	2015-02-17 00:19:41	2015-02-17 01:24:48	4.0	3	1389	3	345	649	17.0
197424	2015-02-13 00:01:59	2015-02-13 00:58:22	4.0	6	3010	4	405	825	12.0
197425	2015-01-24 04:46:08	2015-01-24 05:36:16	4.0	5	1836	3	300	399	39.0
197426	2015-02-01 18:18:15	2015-02-01 19:23:22	1.0	1	1175	1	535	535	7.0
197427	2015-02-08 19:24:33	2015-02-08 20:01:41	1.0	4	2605	4	425	750	20.0

197428 rows × 11 columns

**Handling Missing Values in Continuos Columns

In [24]:

```
print(df['total_onshift_partners'].skew())
print(df['total_busy_partners'].skew())
print(df['total_outstanding_orders'].skew())
```

0.8607580961811951
0.7824625946552558
1.1953217621678773

Inference:

Since all three columns are right-skewed, median imputation is the appropriate method to fill the missing values in these columns.

In [25]:

```
cols = ['total_onshift_partners', 'total_busy_partners', 'total_outstanding_orders']
for col in cols:
    df[col].fillna(df[col].median(), inplace=True)
```

In [26]:

```
df.isnull().sum()
```

Out[26]:

created_at0
actual_delivery_time7
order_protocol0
total_items0
subtotal0
num_distinct_items0
min_item_price0
max_item_price0
total_onshift_partners0
total_busy_partners0
total_outstanding_orders0
dtype: int64

Dropping remaining Null Values

In [27]:

```
df.dropna(inplace=True)
df.isnull().sum()
```

```
Out[27]: created_at          0
actual_delivery_time      0
order_protocol            0
total_items               0
subtotal                 0
num_distinct_items        0
min_item_price            0
max_item_price            0
total_onshift_partners    0
total_busy_partners       0
total_outstanding_orders  0
dtype: int64
```

```
In [28]: df.shape
```

```
Out[28]: (197421, 11)
```



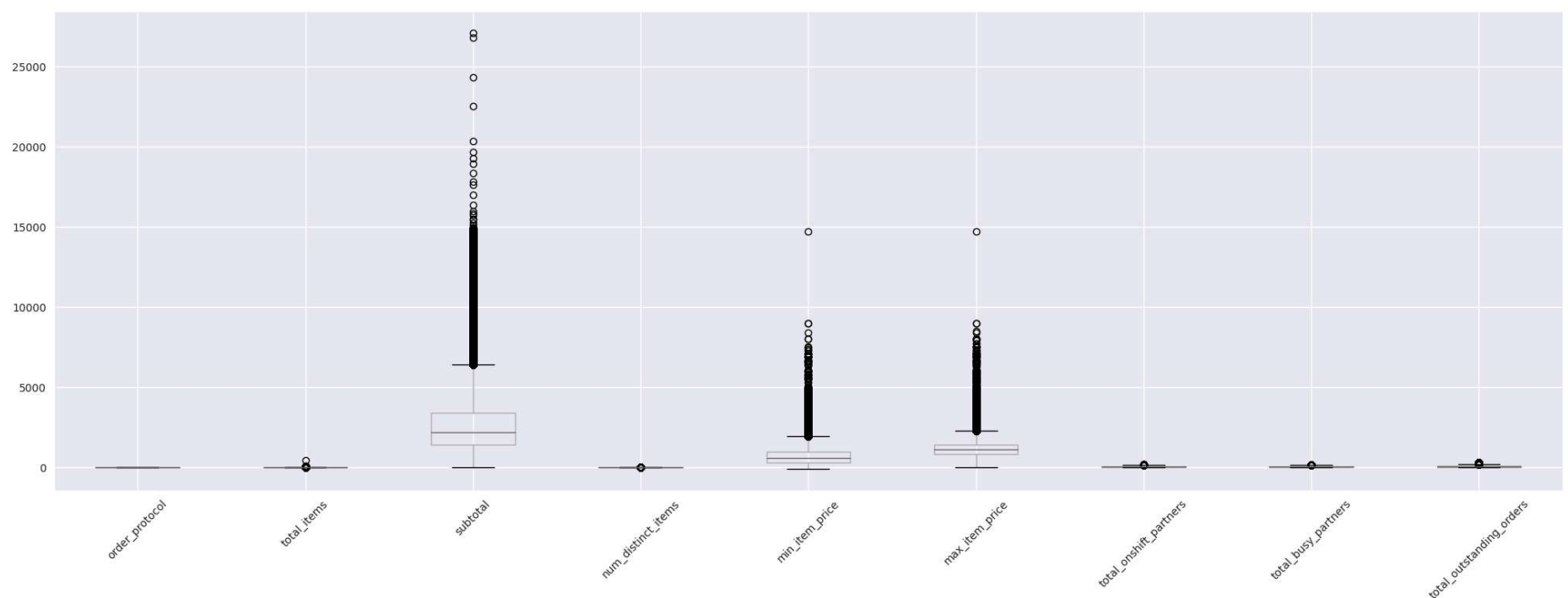
Handling Outliers

Outlier Detection using Box plot

```
In [29]: num_cols = df.select_dtypes(include='number').columns
num_cols
```

```
Out[29]: Index(['order_protocol', 'total_items', 'subtotal', 'num_distinct_items',
               'min_item_price', 'max_item_price', 'total_onshift_partners',
               'total_busy_partners', 'total_outstanding_orders'],
              dtype='object')
```

```
In [30]: df[num_cols].boxplot(rot=45,figsize=(25,8))
plt.show()
```



Handling outliers using LOF method

```
In [31]: len_before = len(df)
len_before
```

```
Out[31]: 197421
```

```
In [32]: from sklearn.neighbors import LocalOutlierFactor
model1 = LocalOutlierFactor()
df['LOF_anomaly_score'] = model1.fit_predict(df[num_cols])
df
```


Out[32]:

	created_at	actual_delivery_time	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners
0	2015-02-06 22:24:17	2015-02-06 23:27:16	1.0	4	3441	4	557	1239	33.0
1	2015-02-10 21:49:25	2015-02-10 22:56:29	2.0	1	1900	1	1400	1400	1.0
2	2015-01-22 20:39:28	2015-01-22 21:09:09	1.0	1	1900	1	1900	1900	1.0
3	2015-02-03 21:21:45	2015-02-03 22:13:00	1.0	6	6900	5	600	1800	1.0
4	2015-02-15 02:40:36	2015-02-15 03:20:26	1.0	3	3900	3	1100	1600	6.0
...	...	...	...	...	...	...	...	...	...
197423	2015-02-17 00:19:41	2015-02-17 01:24:48	4.0	3	1389	3	345	649	17.0
197424	2015-02-13 00:01:59	2015-02-13 00:58:22	4.0	6	3010	4	405	825	12.0
197425	2015-01-24 04:46:08	2015-01-24 05:36:16	4.0	5	1836	3	300	399	39.0
197426	2015-02-01 18:18:15	2015-02-01 19:23:22	1.0	1	1175	1	535	535	7.0
197427	2015-02-08 19:24:33	2015-02-08 20:01:41	1.0	4	2605	4	425	750	20.0

197421 rows × 12 columns

In [33]:

```
outliers = len(df.loc[(df['LOF_anomaly_score']==-1)])
print(f'Number of outliers:{outliers}')
```

Number of outliers:6595

In [34]:

```
df=df.loc[(df['LOF_anomaly_score'] == 1)]
df
```

Out[34]:

	created_at	actual_delivery_time	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners
0	2015-02-06 22:24:17	2015-02-06 23:27:16	1.0	4	3441	4	557	1239	33.0
1	2015-02-10 21:49:25	2015-02-10 22:56:29	2.0	1	1900	1	1400	1400	1.0
2	2015-01-22 20:39:28	2015-01-22 21:09:09	1.0	1	1900	1	1900	1900	1.0
3	2015-02-03 21:21:45	2015-02-03 22:13:00	1.0	6	6900	5	600	1800	1.0
4	2015-02-15 02:40:36	2015-02-15 03:20:26	1.0	3	3900	3	1100	1600	6.0
...	...	...	...	...	...	...	...	...	...
197423	2015-02-17 00:19:41	2015-02-17 01:24:48	4.0	3	1389	3	345	649	17.0
197424	2015-02-13 00:01:59	2015-02-13 00:58:22	4.0	6	3010	4	405	825	12.0
197425	2015-01-24 04:46:08	2015-01-24 05:36:16	4.0	5	1836	3	300	399	39.0
197426	2015-02-01 18:18:15	2015-02-01 19:23:22	1.0	1	1175	1	535	535	7.0
197427	2015-02-08 19:24:33	2015-02-08 20:01:41	1.0	4	2605	4	425	750	20.0

190826 rows × 12 columns

In [35]:

```
df.drop(columns = ['LOF_anomaly_score'], inplace = True)
df
```

Out[35]:

	created_at	actual_delivery_time	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners
0	2015-02-06 22:24:17	2015-02-06 23:27:16	1.0	4	3441	4	557	1239	33.0
1	2015-02-10 21:49:25	2015-02-10 22:56:29	2.0	1	1900	1	1400	1400	1.0
2	2015-01-22 20:39:28	2015-01-22 21:09:09	1.0	1	1900	1	1900	1900	1.0
3	2015-02-03 21:21:45	2015-02-03 22:13:00	1.0	6	6900	5	600	1800	1.0
4	2015-02-15 02:40:36	2015-02-15 03:20:26	1.0	3	3900	3	1100	1600	6.0
...	...	...	...	...	...	...	...	...	...
197423	2015-02-17 00:19:41	2015-02-17 01:24:48	4.0	3	1389	3	345	649	17.0
197424	2015-02-13 00:01:59	2015-02-13 00:58:22	4.0	6	3010	4	405	825	12.0
197425	2015-01-24 04:46:08	2015-01-24 05:36:16	4.0	5	1836	3	300	399	39.0
197426	2015-02-01 18:18:15	2015-02-01 19:23:22	1.0	1	1175	1	535	535	7.0
197427	2015-02-08 19:24:33	2015-02-08 20:01:41	1.0	4	2605	4	425	750	20.0

190826 rows × 11 columns

In [36]:

```
len_after = len(df)
```

In [37]:

```
((len_before-len_after)/ (len_before))*100
```

Out[37]:

```
3.3405767370239237
```

Inference

A total of 3.34% of outlier values have been removed from the dataset using the LOF method.

 Feature Engineering

Extraction datetime Features

In [38]:

```
df['created_at'] = pd.to_datetime(df['created_at'])
df['actual_delivery_time'] = pd.to_datetime(df['actual_delivery_time'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 190826 entries, 0 to 197427
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype
---  -
0   created_at          190826 non-null  datetime64[ns]
1   actual_delivery_time 190826 non-null  datetime64[ns]
2   order_protocol       190826 non-null  float64
3   total_items          190826 non-null  int64
4   subtotal             190826 non-null  int64
5   num_distinct_items   190826 non-null  int64
6   min_item_price       190826 non-null  int64
7   max_item_price       190826 non-null  int64
8   total_onshift_partners 190826 non-null  float64
9   total_busy_partners   190826 non-null  float64
10  total_outstanding_orders 190826 non-null  float64
dtypes: datetime64[ns](2), float64(4), int64(5)
memory usage: 17.5 MB
```

In [39]:

```
df['hour'] = df['created_at'].dt.hour
df['day_of_week'] = df['created_at'].dt.weekday
df.head()
```

Out[39]:

	created_at	actual_delivery_time	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	tota
0	2015-02-06 22:24:17	2015-02-06 23:27:16	1.0	4	3441	4	557	1239	33.0	
1	2015-02-10 21:49:25	2015-02-10 22:56:29	2.0	1	1900	1	1400	1400	1.0	
2	2015-01-22 20:39:28	2015-01-22 21:09:09	1.0	1	1900	1	1900	1900	1.0	
3	2015-02-03 21:21:45	2015-02-03 22:13:00	1.0	6	6900	5	600	1800	1.0	
4	2015-02-15 02:40:36	2015-02-15 03:20:26	1.0	3	3900	3	1100	1600	6.0	

Generating Target Column

To predict delivery time, we calculate it by subtracting the created time from the actual delivery time.

Time Taken = Actual Delivery Time - Created at

In [40]:

```
df['time_taken'] = (df['actual_delivery_time']-df['created_at']).dt.total_seconds()/60
df
```

Out[40]:

	created_at	actual_delivery_time	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	
0	2015-02-06 22:24:17	2015-02-06 23:27:16	1.0	4	3441	4	557	1239	33.0	
1	2015-02-10 21:49:25	2015-02-10 22:56:29	2.0	1	1900	1	1400	1400	1.0	
2	2015-01-22 20:39:28	2015-01-22 21:09:09	1.0	1	1900	1	1900	1900	1.0	
3	2015-02-03 21:21:45	2015-02-03 22:13:00	1.0	6	6900	5	600	1800	1.0	
4	2015-02-15 02:40:36	2015-02-15 03:20:26	1.0	3	3900	3	1100	1600	6.0	
...	...	...	...	...	...	...	...	...	...	...
197423	2015-02-17 00:19:41	2015-02-17 01:24:48	4.0	3	1389	3	345	649	17.0	
197424	2015-02-13 00:01:59	2015-02-13 00:58:22	4.0	6	3010	4	405	825	12.0	
197425	2015-01-24 04:46:08	2015-01-24 05:36:16	4.0	5	1836	3	300	399	39.0	
197426	2015-02-01 18:18:15	2015-02-01 19:23:22	1.0	1	1175	1	535	535	7.0	
197427	2015-02-08 19:24:33	2015-02-08 20:01:41	1.0	4	2605	4	425	750	20.0	

190826 rows × 14 columns

In [41]:

```
df.drop(columns=['created_at','actual_delivery_time'],inplace = True)
df
```

Out[41]:

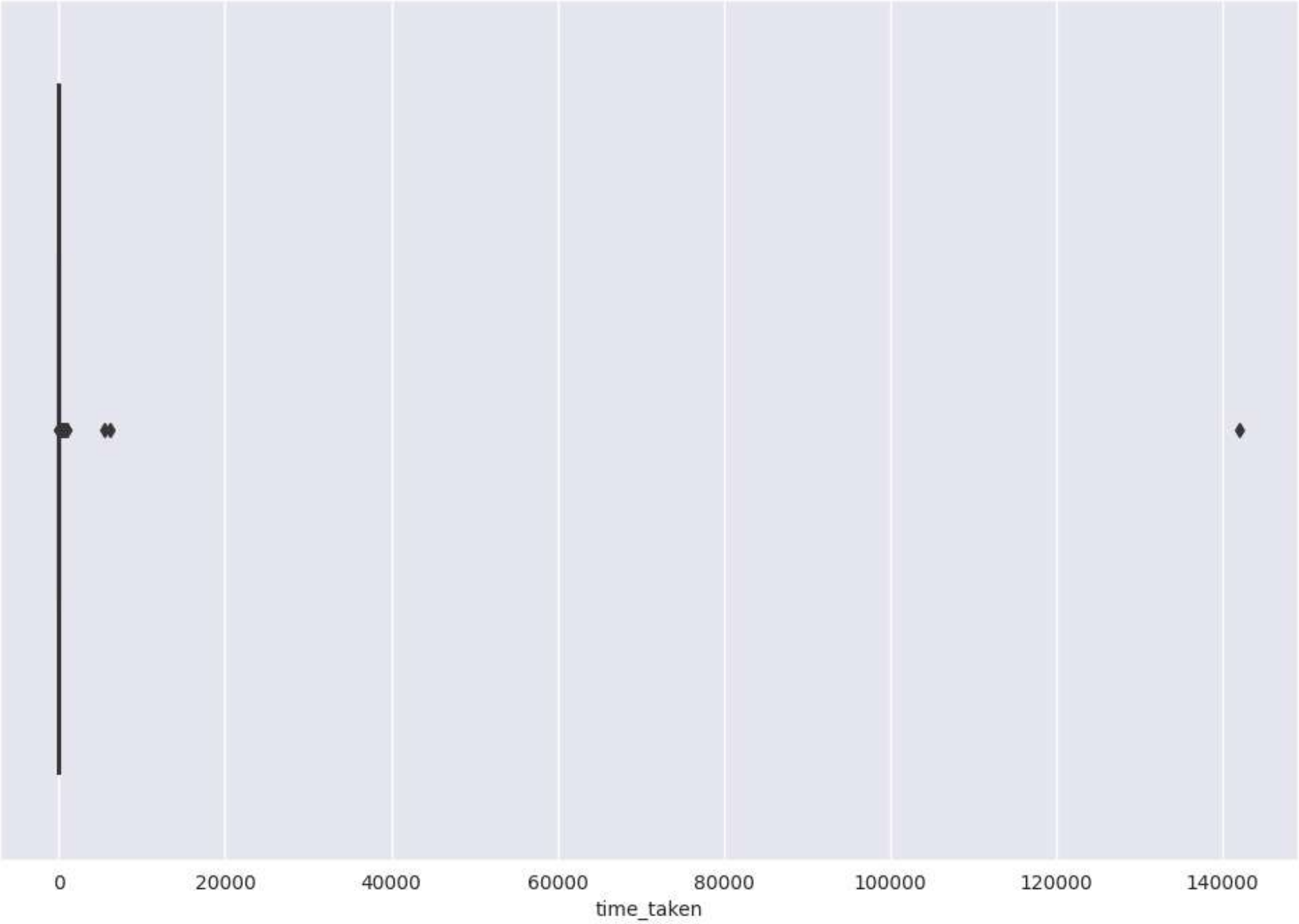
	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outst
0	1.0	4	3441	4	557	1239	33.0	14.0	
1	2.0	1	1900	1	1400	1400	1.0	2.0	
2	1.0	1	1900	1	1900	1900	1.0	0.0	
3	1.0	6	6900	5	600	1800	1.0	1.0	
4	1.0	3	3900	3	1100	1600	6.0	6.0	
...	...	...	...	...	...	...	...	...	...
197423	4.0	3	1389	3	345	649	17.0	17.0	
197424	4.0	6	3010	4	405	825	12.0	11.0	
197425	4.0	5	1836	3	300	399	39.0	41.0	
197426	1.0	1	1175	1	535	535	7.0	7.0	
197427	1.0	4	2605	4	425	750	20.0	20.0	

190826 rows × 12 columns

Checking for outliers in Target Column

In [42]:

```
plt.figure(figsize=(12,8))
sns.boxplot(data = df, x = 'time_taken', color = '#7209B7')
plt.show()
```



Inference:

As we see there are few outliers in target we need to remove these outliers before training the NN model.

Removing Outliers in Target Column

In [43]:

```
# Calculate Q1 (25th percentile) and Q3 (75th percentile)
Q1 = df['time_taken'].quantile(0.25)
Q3 = df['time_taken'].quantile(0.75)
IQR = Q3 - Q1

# Define Lower and upper bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter out extreme values
```

```
df = df[(df['time_taken'] >= lower_bound) & (df['time_taken'] <= upper_bound)]
df
```

Out[43]:

	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outsta
0	1.0	4	3441	4	557	1239	33.0	14.0	
1	2.0	1	1900	1	1400	1400	1.0	2.0	
2	1.0	1	1900	1	1900	1900	1.0	0.0	
3	1.0	6	6900	5	600	1800	1.0	1.0	
4	1.0	3	3900	3	1100	1600	6.0	6.0	
...	...	...	...	...	...	...	...	...	
197423	4.0	3	1389	3	345	649	17.0	17.0	
197424	4.0	6	3010	4	405	825	12.0	11.0	
197425	4.0	5	1836	3	300	399	39.0	41.0	
197426	1.0	1	1175	1	535	535	7.0	7.0	
197427	1.0	4	2605	4	425	750	20.0	20.0	

184768 rows × 12 columns

In [44]:

```
df.isnull().sum()
```

Out[44]:

```
order_protocol      0
total_items         0
subtotal            0
num_distinct_items  0
min_item_price      0
max_item_price      0
total_onshift_partners  0
total_busy_partners  0
total_outstanding_orders  0
hour                0
day_of_week         0
time_taken          0
dtype: int64
```

Splitting Dataset into Training and Test Sets

In [45]:

```
data = df
```

In [46]:

```
X = df.drop(columns=['time_taken'])
y = df['time_taken']
```

In [47]:

```
X
```

Out[47]:

	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outsta
0	1.0	4	3441	4	557	1239	33.0	14.0	
1	2.0	1	1900	1	1400	1400	1.0	2.0	
2	1.0	1	1900	1	1900	1900	1.0	0.0	
3	1.0	6	6900	5	600	1800	1.0	1.0	
4	1.0	3	3900	3	1100	1600	6.0	6.0	
...	...	...	...	...	...	...	...	...	
197423	4.0	3	1389	3	345	649	17.0	17.0	
197424	4.0	6	3010	4	405	825	12.0	11.0	
197425	4.0	5	1836	3	300	399	39.0	41.0	
197426	1.0	1	1175	1	535	535	7.0	7.0	
197427	1.0	4	2605	4	425	750	20.0	20.0	

184768 rows × 11 columns

In [48]:

```
y
```



```
Out[48]: 0      62.983333
1      67.066667
2      29.683333
3      51.250000
4      39.833333
...
197423  65.116667
197424  56.383333
197425  50.133333
197426  65.116667
197427  37.133333
Name: time_taken, Length: 184768, dtype: float64
```

```
In [49]: from sklearn.model_selection import train_test_split
X_dev, X_test, y_dev, y_test = train_test_split(X,y, test_size = 0.3, random_state = 42)
X_train, X_val, y_train, y_val = train_test_split(X_dev,y_dev, test_size = 0.3, random_state = 42)
```

```
In [50]: print(f'Shape of Training Data : {X_train.shape}' )
print(f'Shape of Validation Data: {X_val.shape}')
print(f'Shape of Test Data : {X_test.shape}')
```

Shape of Training Data : (90535, 11)
Shape of Validation Data: (38802, 11)
Shape of Test Data : (55431, 11)

Feature Scaling

```
In [51]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.fit_transform(X_val)
X_test = scaler.fit_transform(X_test)
```

```
In [52]: X_train
```

```
Out[52]: array([[ 1.41239719, -0.48910016,  0.15548868, ...,  2.48988231,
                -0.75028831,  0.8667534 ],
                [ 0.08220495, -0.48910016, -0.79972233, ...,  2.62820989,
                -0.86561841,  0.8667534 ],
                [-1.24798728, -0.48910016, -0.43633285, ..., -0.86950185,
                 1.67164375, -0.61131279],
                ...,
                [ 1.41239719, -0.48910016, -0.26763606, ..., -0.6916521 ,
                -0.51962811,  0.37406467],
                [-1.24798728, -0.09557105, -0.58567099, ...,  2.52940447,
                -0.75028831,  0.8667534 ],
                [ 0.74730107, -0.48910016, -0.38931899, ...,  0.29640206,
                -0.75028831,  1.35944214]])
```

```
In [53]: X_val
```

```
Out[53]: array([[ 0.07877079, -0.91745108, -0.97541725, ..., -0.00793697,
                -0.86103335, -0.60869446],
                [ 1.40564911, -0.50585985, -1.00053942, ..., -0.30372094,
                -0.28367411,  0.37338677],
                [ 1.40564911, -0.09426863,  0.41467607, ...,  2.299178 ,
                -0.7455615 ,  0.86442739],
                ...,
                [ 0.74220995, -0.50585985, -1.08818788, ..., -1.05304033,
                 0.98651621, -1.5907757 ],
                [ 0.07877079, -0.91745108, -0.5595058 , ..., -0.06709376,
                -0.51461781, -0.11765385],
                [-1.24810753,  0.72891383,  0.06966496, ..., -0.38259666,
                -0.86103335,  0.37338677]])
```

```
In [54]: X_test
```

```
Out[54]: array([[ 1.4100041 , -0.90565717, -0.73045639, ..., -0.59202661,
                 1.2148872 , -1.09802391],
                [ 1.4100041 ,  3.15414491,  0.06234062, ..., -1.12635014,
                -0.86105146, -1.09802391],
                [-1.25352518, -0.09369675, -0.59415094, ..., -0.96803206,
                 1.6762069 , -0.60650284],
                ...,
                [ 0.74412178, -0.90565717, -0.05226723, ..., -0.71076517,
                 1.2148872 , -0.11498176],
                [ 0.07823946, -0.49967696,  0.03174144, ...,  0.27872284,
                -0.51506169,  0.8680604 ],
                [ 1.4100041 ,  1.93620429,  3.78208859, ...,  0.15998428,
                -0.74572154, -1.09802391]])
```

EDA

Univariate Analysis

Distribution of 'order_protocl'

In [55]: data

Out[55]:

	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outst
0	1.0	4	3441	4	557	1239	33.0	14.0	
1	2.0	1	1900	1	1400	1400	1.0	2.0	
2	1.0	1	1900	1	1900	1900	1.0	0.0	
3	1.0	6	6900	5	600	1800	1.0	1.0	
4	1.0	3	3900	3	1100	1600	6.0	6.0	
...	...	...	...	...	...	...	...	...	...
197423	4.0	3	1389	3	345	649	17.0	17.0	
197424	4.0	6	3010	4	405	825	12.0	11.0	
197425	4.0	5	1836	3	300	399	39.0	41.0	
197426	1.0	1	1175	1	535	535	7.0	7.0	
197427	1.0	4	2605	4	425	750	20.0	20.0	

184768 rows × 12 columns

```
In [56]: labels=data['order_protocol'].value_counts().index
values=data['order_protocol'].value_counts()

#create pie chart
# Create pie chart
#colors = ["#2c699a", "#048ba8", "#0db39e", "#83e377", "#b9e769", "#efea5a", "#f1c453"]
colors = bo

pie_chart = go.Figure(go.Pie(labels=labels, values=values))
pie_chart.update_traces(hoverinfo='label+value', textinfo='percent', textfont_size=20,
                        marker=dict(colors=colors, line=dict(color='#000000', width=2)))

# Create bar chart
bar_chart = go.Figure(go.Bar(x=labels, y=values, marker_color=colors))

# Create subplots
fig = sp.make_subplots(rows=1, cols=2, column_widths=[0.5, 0.5], specs=[[{"type": "bar"}, {"type": "pie"}]],
                      subplot_titles=("Bar Chart", "Pie Chart"))

# Add charts to subplots
fig.add_trace(bar_chart.data[0], row=1, col=1)
fig.add_trace(pie_chart.data[0], row=1, col=2)

# Update Layout
fig.update_layout(
    showlegend=False,
    title=dict(text="Distribution of Order Protocol", font=dict(size=16)),
    xaxis=dict(title=dict(text='Order Protocol', font=dict(size=16)), tickfont=dict(size=14)),
    yaxis=dict(title=dict(text='Number of Orders', font=dict(size=16)), tickfont=dict(size=14))
)

# Show subplots
fig.show()
```

Insights:

- 27.9% of orders were placed using Order Protocol 1.
- 12.2% of orders followed Order Protocol 2.
- 27.2% of orders were processed through Order Protocol 3.
- 9.69% of orders used Order Protocol 4.
- 22.6% of orders were placed via Order Protocol 5.
- 0.4% of orders followed Order Protocol 6.
- The remaining 0.009% of orders were placed under Order Protocol 7.

Distribution of 'total_items','subtotal' and 'num_distinct_items'

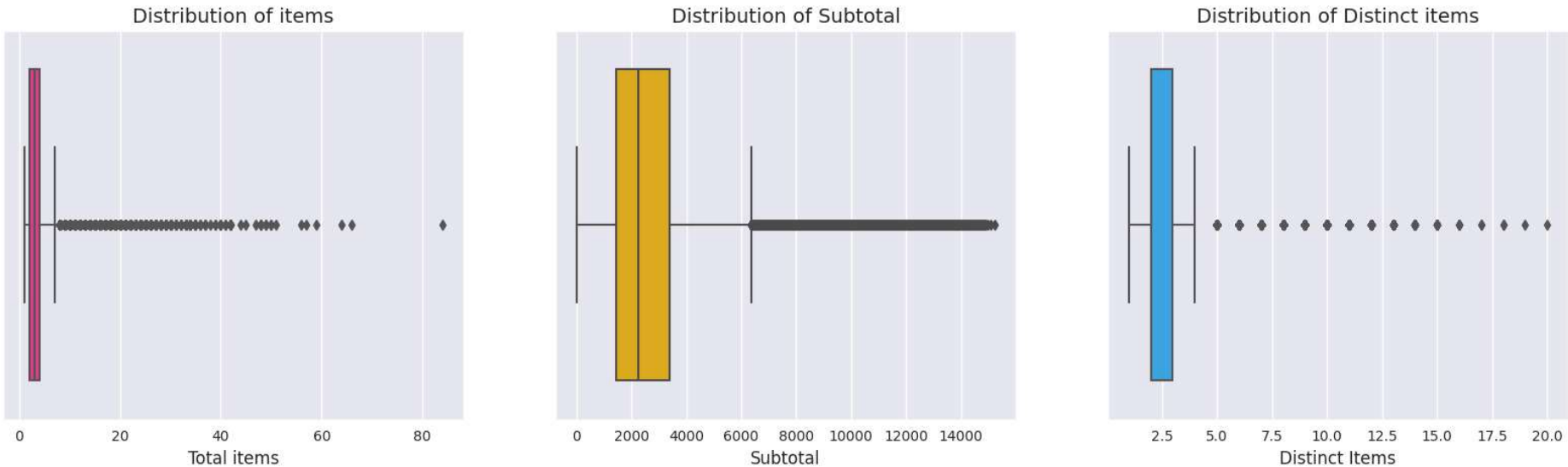
```
In [57]: plt.figure(figsize=(20,5))

# Boxplot for total_items column
plt.subplot(1,3,1)
sns.boxplot(data=data,x='total_items',color='#ff228a')
plt.xlabel('Total items',fontsize=12)
plt.title('Distribution of items',fontsize=14)

#Boxplot for subtotal column
plt.subplot(1,3,2)
sns.boxplot(data=df,x='subtotal',color='#ffb703')
plt.xlabel('Subtotal',fontsize=12)
plt.title('Distribution of Subtotal',fontsize=14)

#Boxplot for num_destinct_items
plt.subplot(1,3,3)
sns.boxplot(data=df,x='num_distinct_items',color='#20b1fd')
plt.xlabel('Distinct Items',fontsize=12)
plt.title('Distribution of Distinct items',fontsize=14)

plt.show()
```



Insights

- Most orders have a small number of total items, but a few orders contain significantly higher quantities, creating outliers.
- The subtotal distribution is right-skewed, with most orders having lower values and a few high-value transactions.
- The number of distinct items per order is generally low, with some orders containing a wide variety of unique items.
- All three features exhibit right-skewed distributions, indicating the presence of extreme values that could impact analysis and decision-making.

Distribution of total_onshift_partners, total_busy_partners and total_outstanding_orders

```
In [58]: colors = ['#9FA0FF', '#DAB6FC', '#86CB92']

# Create histogram for GRE Score
hist_1 = go.Histogram(x=data['total_onshift_partners'], marker_color=colors[0])

# Create histogram for TOEFL Score
hist_2 = go.Histogram(x=data['total_busy_partners'], marker_color=colors[1])

# Create histogram for CGPA
hist_3 = go.Histogram(x=data['total_outstanding_orders'], marker_color=colors[2])

# Create subplots
fig = make_subplots(rows=1, cols=3, subplot_titles=(" Total Onshift Partners", " Total Busy Partners"," Total Outstanding Orders'

# Add histograms to subplots
fig.add_trace(hist_1, row=1, col=1)
fig.add_trace(hist_2, row=1, col=2)
fig.add_trace(hist_3, row=1, col=3)

# Update Layout
fig.update_layout(barmode='overlay', height=600, width=1400,showlegend=False)

# Show plot
fig.show()
```

Insights:

- Total Onshift Partners & Total Busy Partners:
 - Most values are concentrated in the 0-50 range, with a sharp peak suggesting a common frequent value.
 - The long tail indicates that higher values (up to 150) exist but are rare.
 - The similar pattern in both distributions suggests that the number of available partners closely follows the number of busy partners.
 - The sharp spike may indicate data irregularities or a specific operational pattern.
- Total Outstanding Orders:

- The majority of outstanding orders are in the 0-50 range, similar to partner distributions.
- A sharp spike is present, which may indicate data logging issues or a highly recurring value.
- The long tail extends beyond 200, meaning that in some instances, outstanding orders pile up significantly.

Hourly order trend

```
In [59]: hour_df=df.groupby(df['hour']).agg(total_orders=('hour','count')).reset_index()  
hour_df
```

Out[59]:

	hour	total_orders
0	0	11967
1	1	26700
2	2	34174
3	3	25086
4	4	14310
5	5	6656
6	6	1337
7	7	10
8	14	32
9	15	482
10	16	1956
11	17	3181
12	18	4806
13	19	12721
14	20	14653
15	21	10749
16	22	8304
17	23	7644

```
In [60]: fig = px.line(hour_df, x='hour', y='total_orders', markers=True)  
fig.update_xaxes(tickvals=list(range(25)))  
fig.update_layout(title='Hourly Order Trend',  
                  xaxis_title='Hours',  
                  yaxis_title='Total Orders')  
fig.show()
```

Insights:

- The number of orders shows an increasing trend from 12 AM to 2 AM, reaching its peak at 2 AM.
- A sharp decline in orders is observed between 2 AM and 7 AM.
- A flat trend between 7 AM and 2 PM suggests no orders were placed, possibly indicating a maintenance period for porters.

- Orders start rising again from 2 PM to 8 PM, followed by a decline between 8 PM and 11 PM.

Daily order Trend

```
In [61]: daily_df=df.groupby(df['day_of_week']).agg(total_orders=('day_of_week','count')).reset_index()
daily_df
```

Out[61]:

	day_of_week	total_orders
0	0	24511
1	1	22590
2	2	23105
3	3	24096
4	4	26586
5	5	32346
6	6	31534

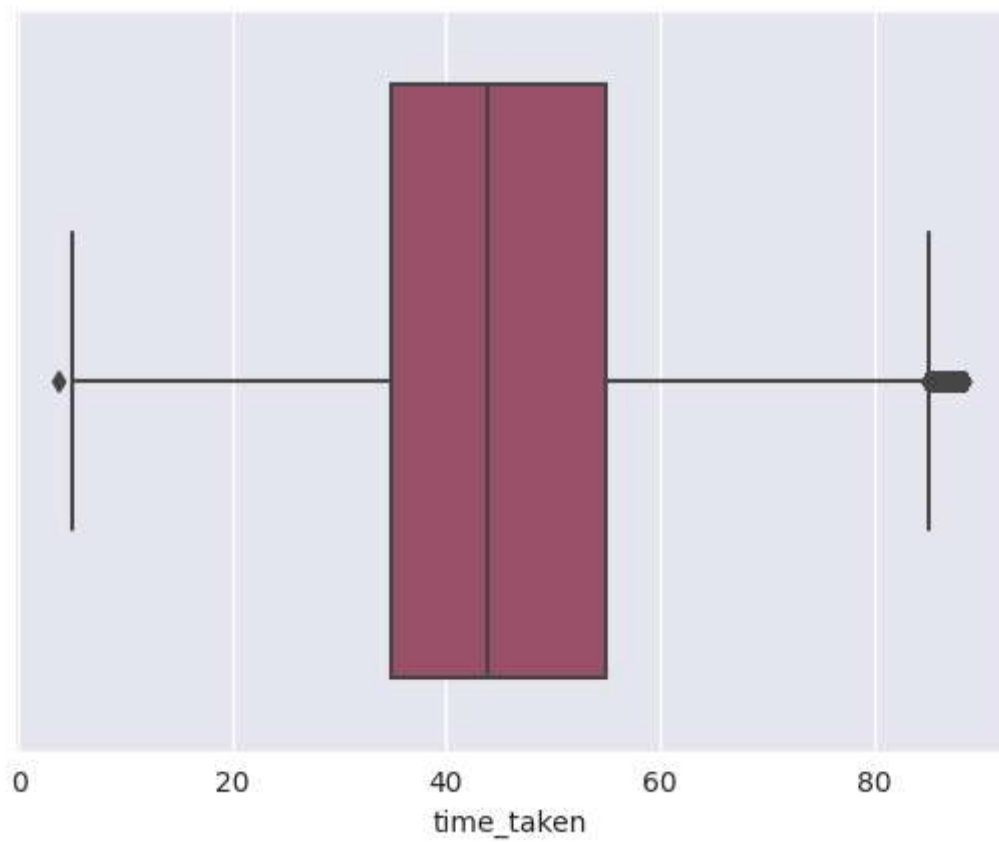
```
In [62]: fig = px.line(daily_df, x='day_of_week', y='total_orders', markers=True )
fig.update_xaxes(tickvals=list(range(25)))
fig.update_layout(title='Daily Order Trend',
                  xaxis_title='Day',
                  yaxis_title='Total Orders')
fig.show()
```

Insights:

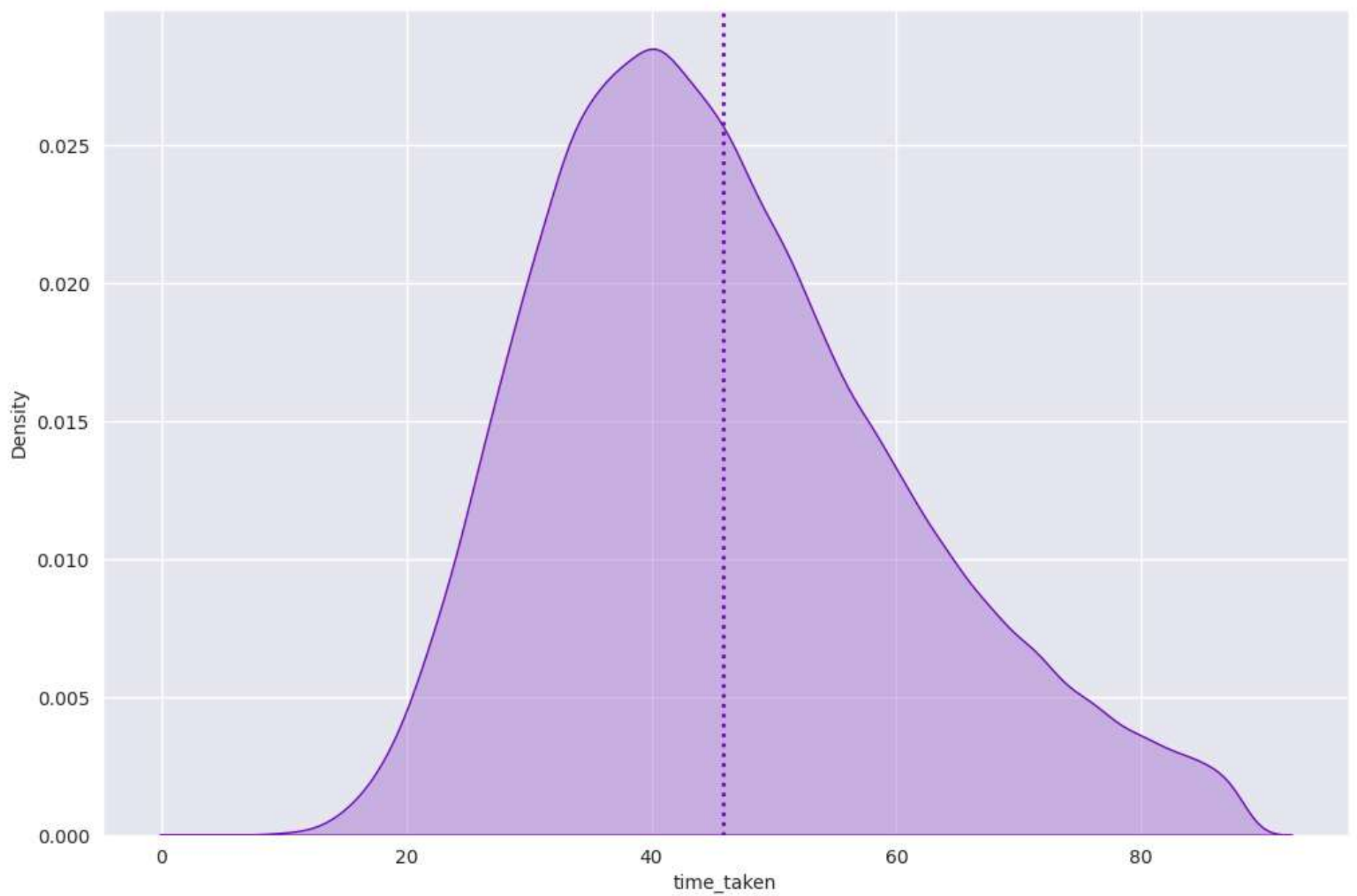
- Orders started relatively high on Day 0 but dropped significantly on Day 1.
- A slight increase is observed on Day 2, followed by a steady rise from Day 3 onward.
- The highest number of orders was recorded on Day 5.
- There is a slight decline on Day 6, but the overall trend suggests increasing order activity towards the end of the week.

Distribution of Target Variable

```
In [63]: sns.boxplot(data = data , x='time_taken')
plt.show()
```

```
In [64]: plt.figure(figsize = (12,8))
sns.kdeplot(df['time_taken'], shade = True, color = '#7209B7')
plt.axvline(df['time_taken'].mean(), color='#7209B7',linestyle=':', linewidth=2)
plt.show()
```



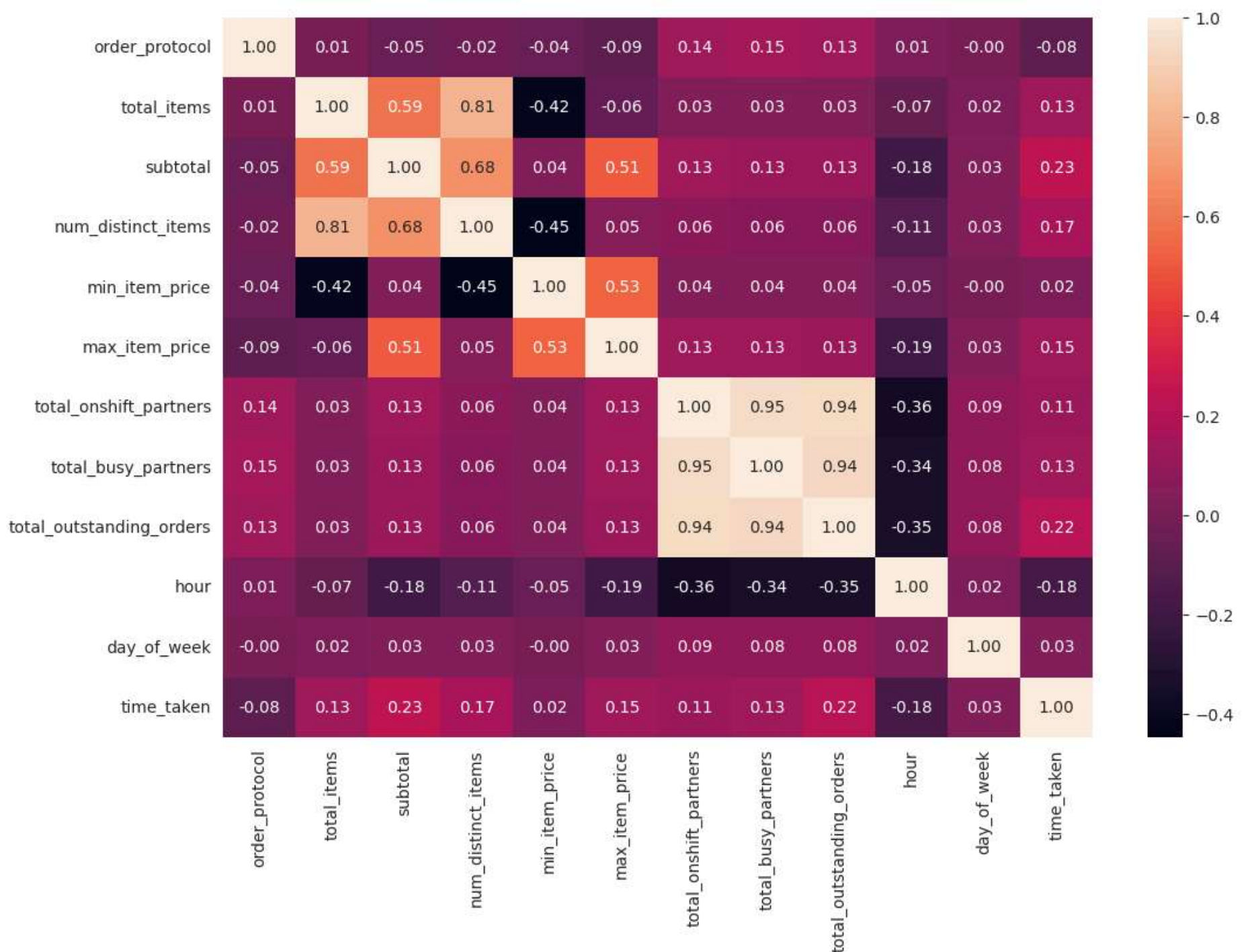
Insights:

- The average delivery time taken by a porter is approximately 43 minutes.
- The target column exhibits a slight right skew.

Bivariate Analysis

Correlation Heatmap

```
In [65]: plt.figure(figsize = (12,8))
sns.heatmap(df.corr(),annot=True,cmap='rocket',fmt='.2f')
plt.show()
```



In []:

Model Training and Building

🔧 Hyperparameter Tuning using Keras Tuner

Define the model

```
In [66]: import keras_tuner as kt

def build_model(hp):
    model = keras.Sequential()

    #Define input Layer
    model.add(keras.layers.Dense(
        units = hp.Int('units', min_value = 32, max_value = 512 , step = 32), # Tune neurons 32-512
    ))
    # Apply leaky relu activation funtion
    model.add(keras.layers.LeakyReLU(alpha=0.1))

    # Batch Normalization after input Layer
    model.add(keras.layers.BatchNormalization())

    # Hidden Layers with Batch Normalization & optional dropout
    for i in range(hp.Int('num_layers',1,5)): # Tune number of hidden layers : (1 to 5)
        model.add(keras.layers.Dense(
            units = hp.Int(f'units_{i}', min_value = 32, max_value = 512, step = 32),
        ))

        model.add(keras.layers.LeakyReLU(alpha=0.1))
        model.add(keras.layers.BatchNormalization())

    # Apply Dropout only if hp.Boolean is True
    if hp.Boolean(f'apply_dropout_{i}'):
        model.add(keras.layers.Dropout(hp.Float(f'dropout_{i}', min_value = 0.1, max_value = 0.5 , step = 0.1)))

    # Output Layer : (Regression : Single Neuron, Linear Activation)
    model.add(keras.layers.Dense(1, activation = 'linear'))

    # Compile model with built-in R2 metric
    model.compile(
```

```

optimizer = keras.optimizers.Adam(
    learning_rate = hp.Float('learning_rate', min_value = 0.0001, max_value = 0.01, sampling = 'log')
),
loss = 'mse', # mean squared error for regression tasks
metrics = [keras.metrics.R2Score()] # R2 metric to evaluate performance on validation data

)

return model

```

Hyperparameter Tuning using Bayesian optimization

```

In [67]: tuner = kt.BayesianOptimization(
    build_model,
    objective = kt.Objective('val_r2_score', direction = 'max'),# Maximize R2 Score
    max_trials = 20, # Number of hyperparameter combinations to test
    executions_per_trial = 2 , # runs each model twice for stability
    directory = '/kaggle/working/',
    project_name = 'Porter : NN Regression'
)

```

Start Hyperparameter Tuning

```

In [68]: tuner.search(
    X_train, y_train, epochs = 20, validation_data = (X_val,y_val), batch_size=512
)

```

Trial 20 Complete [00h 00m 51s]
val_r2_score: 0.22247031331062317

Best val_r2_score So Far: 0.24619296193122864
Total elapsed time: 00h 13m 41s

Retrive Best Hyperparameters

```

In [69]: best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

print(f"""
Best number of units: {best_hps.get('units')}
Best number of layers: {best_hps.get('num_layers')}
Best learning rate: {best_hps.get('learning_rate')}
""")

for i in range(best_hps.get('num_layers')):
    if best_hps.get(f'apply_dropout_{i}'):
        print(f"Dropout applied in layer {i} with rate {best_hps.get(f'dropout_{i}')}")
    else:
        print(f"No Dropout in layer {i}")

```

Best number of units: 224
Best number of layers: 3
Best learning rate: 0.006610542448699214

Dropout applied in layer 0 with rate 0.2
Dropout applied in layer 1 with rate 0.2
Dropout applied in layer 2 with rate 0.30000000000000004

Train the Best Model

```

In [70]: best_model = tuner.hypermodel.build(best_hps)
history = best_model.fit(X_train, y_train, epochs = 50 , batch_size = 512, validation_data = (X_val,y_val))

```


Epoch 1/50
177/177  8s 19ms/step - loss: 1049.7002 - r2_score: -3.8083 - val_loss: 179.7181 - val_r2_score: 0.1736
Epoch 2/50
177/177  1s 3ms/step - loss: 188.4011 - r2_score: 0.1317 - val_loss: 176.5751 - val_r2_score: 0.1881
Epoch 3/50
177/177  1s 3ms/step - loss: 185.2897 - r2_score: 0.1545 - val_loss: 172.4494 - val_r2_score: 0.2070
Epoch 4/50
177/177  1s 3ms/step - loss: 180.0428 - r2_score: 0.1697 - val_loss: 176.8416 - val_r2_score: 0.1868
Epoch 5/50
177/177  1s 3ms/step - loss: 176.3495 - r2_score: 0.1867 - val_loss: 170.3888 - val_r2_score: 0.2165
Epoch 6/50
177/177  1s 3ms/step - loss: 175.4228 - r2_score: 0.1888 - val_loss: 170.6527 - val_r2_score: 0.2153
Epoch 7/50
177/177  1s 3ms/step - loss: 171.4171 - r2_score: 0.2047 - val_loss: 168.0497 - val_r2_score: 0.2273
Epoch 8/50
177/177  1s 3ms/step - loss: 171.9145 - r2_score: 0.2106 - val_loss: 166.5827 - val_r2_score: 0.2340
Epoch 9/50
177/177  1s 3ms/step - loss: 171.0158 - r2_score: 0.2173 - val_loss: 165.8492 - val_r2_score: 0.2374
Epoch 10/50
177/177  1s 3ms/step - loss: 169.5049 - r2_score: 0.2187 - val_loss: 165.6583 - val_r2_score: 0.2382
Epoch 11/50
177/177  1s 3ms/step - loss: 168.6750 - r2_score: 0.2227 - val_loss: 168.5814 - val_r2_score: 0.2248
Epoch 12/50
177/177  1s 3ms/step - loss: 167.9926 - r2_score: 0.2216 - val_loss: 168.4077 - val_r2_score: 0.2256
Epoch 13/50
177/177  1s 3ms/step - loss: 167.4341 - r2_score: 0.2263 - val_loss: 165.5409 - val_r2_score: 0.2388
Epoch 14/50
177/177  1s 3ms/step - loss: 168.7444 - r2_score: 0.2194 - val_loss: 168.4506 - val_r2_score: 0.2254
Epoch 15/50
177/177  1s 3ms/step - loss: 167.9000 - r2_score: 0.2254 - val_loss: 167.6576 - val_r2_score: 0.2291
Epoch 16/50
177/177  1s 3ms/step - loss: 167.1943 - r2_score: 0.2283 - val_loss: 166.8346 - val_r2_score: 0.2328
Epoch 17/50
177/177  1s 3ms/step - loss: 166.7579 - r2_score: 0.2328 - val_loss: 164.9429 - val_r2_score: 0.2415
Epoch 18/50
177/177  1s 3ms/step - loss: 165.1527 - r2_score: 0.2318 - val_loss: 164.5174 - val_r2_score: 0.2435
Epoch 19/50
177/177  1s 3ms/step - loss: 166.4207 - r2_score: 0.2281 - val_loss: 163.8973 - val_r2_score: 0.2463
Epoch 20/50
177/177  1s 3ms/step - loss: 164.3853 - r2_score: 0.2369 - val_loss: 164.3264 - val_r2_score: 0.2444
Epoch 21/50
177/177  1s 3ms/step - loss: 165.6597 - r2_score: 0.2360 - val_loss: 165.0731 - val_r2_score: 0.2409
Epoch 22/50
177/177  1s 3ms/step - loss: 166.8883 - r2_score: 0.2333 - val_loss: 164.2664 - val_r2_score: 0.2447
Epoch 23/50
177/177  1s 3ms/step - loss: 165.6518 - r2_score: 0.2328 - val_loss: 168.3119 - val_r2_score: 0.2260
Epoch 24/50
177/177  1s 3ms/step - loss: 165.2957 - r2_score: 0.2331 - val_loss: 164.7110 - val_r2_score: 0.2426
Epoch 25/50
177/177  1s 3ms/step - loss: 164.1541 - r2_score: 0.2452 - val_loss: 163.9146 - val_r2_score: 0.2463
Epoch 26/50
177/177  1s 3ms/step - loss: 165.3348 - r2_score: 0.2393 - val_loss: 163.1830 - val_r2_score: 0.2496
Epoch 27/50
177/177  1s 3ms/step - loss: 164.1961 - r2_score: 0.2417 - val_loss: 165.0998 - val_r2_score: 0.2408
Epoch 28/50
177/177  1s 3ms/step - loss: 164.2494 - r2_score: 0.2398 - val_loss: 163.9237 - val_r2_score: 0.2462
Epoch 29/50
177/177  1s 3ms/step - loss: 166.0402 - r2_score: 0.2393 - val_loss: 164.2788 - val_r2_score: 0.2446
Epoch 30/50
177/177  1s 3ms/step - loss: 163.1025 - r2_score: 0.2447 - val_loss: 164.6427 - val_r2_score: 0.2429
Epoch 31/50
177/177  1s 3ms/step - loss: 163.1636 - r2_score: 0.2438 - val_loss: 163.1351 - val_r2_score: 0.2499
Epoch 32/50
177/177  1s 3ms/step - loss: 163.5167 - r2_score: 0.2416 - val_loss: 164.1235 - val_r2_score: 0.2453
Epoch 33/50
177/177  1s 3ms/step - loss: 164.7149 - r2_score: 0.2406 - val_loss: 164.2355 - val_r2_score: 0.2448
Epoch 34/50
177/177  1s 3ms/step - loss: 163.9882 - r2_score: 0.2470 - val_loss: 164.2583 - val_r2_score: 0.2447
Epoch 35/50
177/177  1s 3ms/step - loss: 163.7724 - r2_score: 0.2478 - val_loss: 162.7428 - val_r2_score: 0.2517
Epoch 36/50
177/177  1s 3ms/step - loss: 163.9256 - r2_score: 0.2426 - val_loss: 165.2911 - val_r2_score: 0.2399
Epoch 37/50
177/177  1s 3ms/step - loss: 163.6659 - r2_score: 0.2429 - val_loss: 163.1826 - val_r2_score: 0.2496
Epoch 38/50
177/177  1s 3ms/step - loss: 162.7568 - r2_score: 0.2455 - val_loss: 163.2958 - val_r2_score: 0.2491
Epoch 39/50
177/177  1s 3ms/step - loss: 161.7279 - r2_score: 0.2490 - val_loss: 164.5928 - val_r2_score: 0.2431
Epoch 40/50
177/177  1s 3ms/step - loss: 163.4823 - r2_score: 0.2468 - val_loss: 164.1170 - val_r2_score: 0.2453
Epoch 41/50
177/177  1s 3ms/step - loss: 163.0679 - r2_score: 0.2495 - val_loss: 163.6490 - val_r2_score: 0.2475
Epoch 42/50
177/177  1s 3ms/step - loss: 162.4168 - r2_score: 0.2498 - val_loss: 163.8829 - val_r2_score: 0.2464
Epoch 43/50
177/177  1s 3ms/step - loss: 164.1023 - r2_score: 0.2446 - val_loss: 164.9312 - val_r2_score: 0.2416
Epoch 44/50
177/177  1s 3ms/step - loss: 163.5244 - r2_score: 0.2482 - val_loss: 162.4162 - val_r2_score: 0.2532
Epoch 45/50
177/177  1s 3ms/step - loss: 163.3179 - r2_score: 0.2497 - val_loss: 162.6660 - val_r2_score: 0.2520
Epoch 46/50
177/177  1s 3ms/step - loss: 163.1151 - r2_score: 0.2512 - val_loss: 162.7673 - val_r2_score: 0.2515
Epoch 47/50

177/177 1s 3ms/step - loss: 162.5166 - r2_score: 0.2531 - val_loss: 162.6575 - val_r2_score: 0.2520
Epoch 48/50
177/177 1s 3ms/step - loss: 162.1092 - r2_score: 0.2526 - val_loss: 162.6766 - val_r2_score: 0.2520
Epoch 49/50
177/177 1s 3ms/step - loss: 163.3271 - r2_score: 0.2479 - val_loss: 165.0871 - val_r2_score: 0.2409
Epoch 50/50
177/177 1s 3ms/step - loss: 161.5627 - r2_score: 0.2554 - val_loss: 163.4434 - val_r2_score: 0.2484

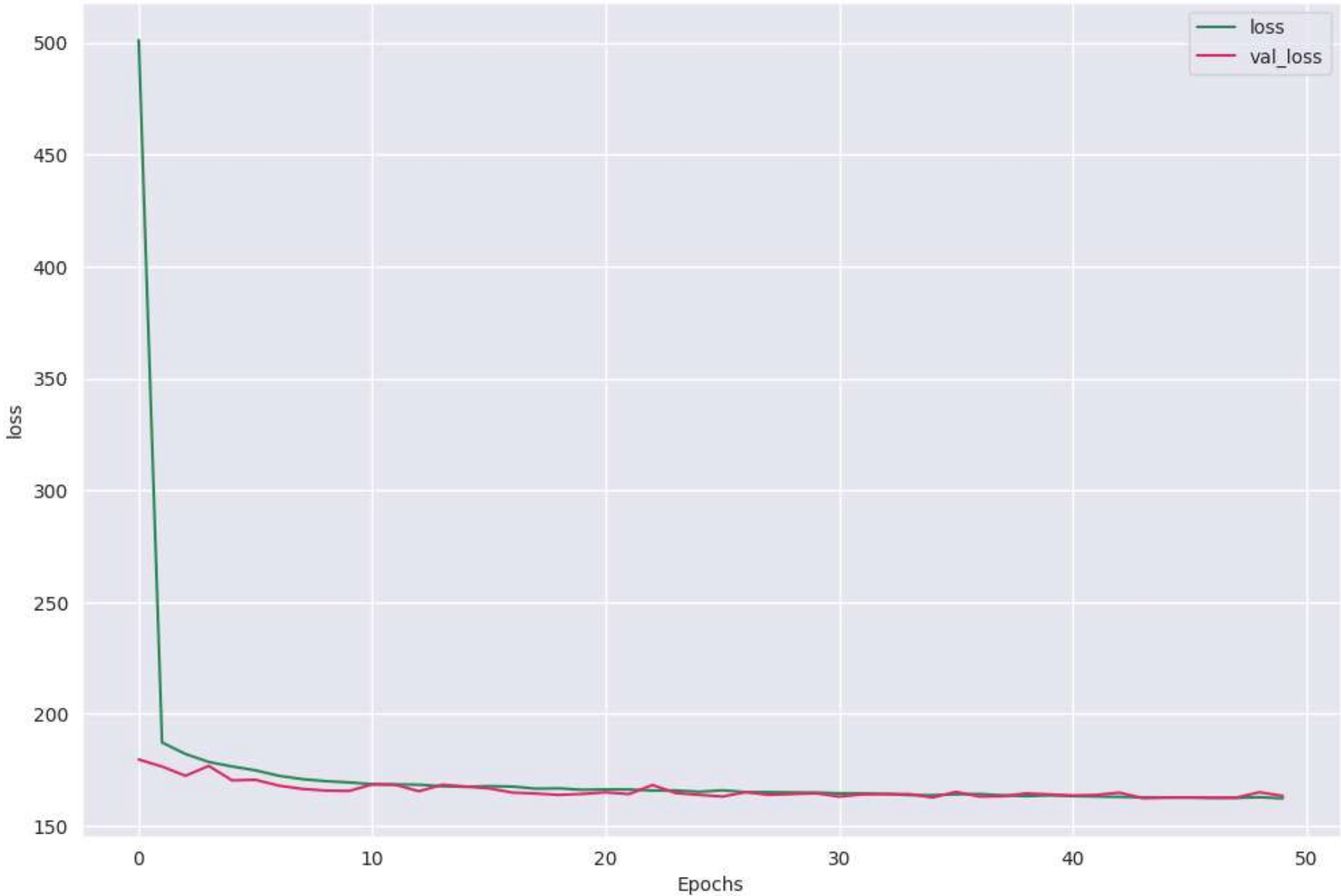
Final Predictions

In [71]: y_pred = best_model.predict(X_test)
y_pred

Out[71]: 1733/1733 3s 1ms/step
array([[38.76761],
 [55.703648],
 [40.68727],
 ...,
 [51.678974],
 [44.28278],
 [59.106403]], dtype=float32)

Evaluating Model performance

In [72]: def plot_history(history, key):
plt.figure(figsize = (12,8))
plt.plot(history.history[key],color = 'seagreen')
plt.plot(history.history['val_'+key],color = '#DB3069')
plt.xlabel('Epochs')
plt.ylabel(key)
plt.legend([key, 'val_'+key])
plt.show()
plot_history(history,'loss')



📈 Performance Metrics

In [73]: from sklearn.metrics import r2_score

def adjusted_r2_score(y_true, y_pred, n, p):
r2 = r2_score(y_true, y_pred)
return 1 - ((1 - r2) * (n - 1) / (n - p - 1))

In [74]: n = X_test.shape[0] # Number of samples
p = X_test.shape[1] # Number of features

In [75]: from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
print(f'MSE : {mean_squared_error(y_test,y_pred)}')
print(f'RMSE : {np.sqrt(mean_squared_error(y_test,y_pred))}')
print(f'MAE : {mean_absolute_error(y_test,y_pred)}')

```
print(f'R2 Score : {r2_score(y_test,y_pred)}')
print(f'Adj R2 Score : {adjusted_r2_score(y_test,y_pred,n,p)}')
```

MSE : 165.23497004250402
RMSE : 12.854375521296397
MAE : 10.233696077631746
R2 Score : 0.24704735597356664
Adj R2 Score : 0.24689790399709122

Insights

- 27.9% of orders were placed using Order Protocol 1.
- 12.2% of orders followed Order Protocol 2.
- 27.2% of orders were processed through Order Protocol 3.
- 9.69% of orders used Order Protocol 4.
- 22.6% of orders were placed via Order Protocol 5.
- 0.4% of orders followed Order Protocol 6.
- The remaining 0.009% of orders were placed under Order Protocol 7.
- Most orders have a small number of total items, but a few orders contain significantly higher quantities, creating outliers.
- The subtotal distribution is right-skewed, with most orders having lower values and a few high-value transactions.
- The number of distinct items per order is generally low, with some orders containing a wide variety of unique items.
- Total Onshift Partners & Total Busy Partners:
 - Most values are concentrated in the 0-50 range, with a sharp peak suggesting a common frequent value.
 - The long tail indicates that higher values (up to 150) exist but are rare.
 - The similar pattern in both distributions suggests that the number of available partners closely follows the number of busy partners.
 - The sharp spike may indicate data irregularities or a specific operational pattern.
- Total Outstanding Orders:
 - The majority of outstanding orders are in the 0-50 range, similar to partner distributions.
 - A sharp spike is present, which may indicate data logging issues or a highly recurring value.
 - The long tail extends beyond 200, meaning that in some instances, outstanding orders pile up significantly.
- The number of orders shows an increasing trend from 12 AM to 2 AM, reaching its peak at 2 AM.
- A sharp decline in orders is observed between 2 AM and 7 AM.
- A flat trend between 7 AM and 2 PM suggests no orders were placed, possibly indicating a maintenance period for porters.
- Orders start rising again from 2 PM to 8 PM, followed by a decline between 8 PM and 11 PM.
- Orders started relatively high on Day 0 but dropped significantly on Day 1.
- A slight increase is observed on Day 2, followed by a steady rise from Day 3 onward.
- The highest number of orders was recorded on Day 5.
- There is a slight decline on Day 6, but the overall trend suggests increasing order activity towards the end of the week.
- The average delivery time taken by a porter is approximately 43 minutes.
- The target column exhibits a slight right skew.

Recommendations

1. Optimize Workforce Allocation:

- Since partner availability closely follows demand, ensure adequate staffing during peak hours (12 AM - 2 AM and 2 PM - 8 PM).
 - Address potential bottlenecks by having on-call partners during high-demand periods.

2. Investigate and Address Data Anomalies:

- The sharp spikes in partner availability and outstanding orders suggest possible data irregularities or operational inefficiencies.
- Conduct a thorough review of order logging processes to ensure accuracy and avoid recurring issues.

3. Improve Order Processing Efficiency:

- Since a few orders contain significantly higher item counts, consider implementing batch processing or priority handling for large orders.
- Reduce outliers by optimizing inventory management and order fulfillment workflows.

4. Adjust Service Hours for Better Order Fulfillment:

- The flat trend between 7 AM - 2 PM suggests a possible service gap.
- If feasible, explore options to extend porter availability or introduce scheduled order fulfillment to maintain order consistency.

5. Enhance High-Value Order Handling:

- The right-skewed subtotal distribution indicates a few high-value transactions.
- Implement strategies such as VIP customer prioritization, dedicated delivery partners, or premium service options to enhance service for high-value orders.